NETFLOW ANALYZER

End-to-End Network Traffic Analysis Pipeline

PCAP Capture · Flow Reconstruction · L3/L4 Features · Machine Learning · Streamlit Dashboard

96.97%	96.89%	4,025	3,999
Test accuracy	Weighted F1	Dashboard flows	Anomalous flows

Final Technical Report

Computer Networks and Communications

Author: Ángel Pérez Castro

Date: 28 May 2026

Scope statement

This report evaluates network behavior using only Layer 3 and Layer 4 metadata. Packet payloads are intentionally excluded to preserve privacy and keep the method applicable to encrypted traffic.

Table of Contents

Table 1. Report structure.

Table 2. Figure placement guide used in this report.

Section	Figure	Asset	Why it is included
Architecture	Figure 1	System architecture and data flow	Used to explain the Docker lab and the full pipeline.
Dataset	Figure 2	Class distribution	Shows class balance and moderate imbalance.
Features	Figure 3	PCA 2D class projection	Best visual evidence that engineered features create separable traffic groups.
Features	Figure 4	Correlation heatmap	Shows feature relationships before ML.
ML	Figure 5	PCA / K-Means visualisation	Shows cluster structure in reduced feature space.
Evaluation	Figure 6	Confusion matrix	Shows actual classification errors.
Evaluation	Figure 7	F1-score chart	Highlights per-class performance.
Evaluation	Figure 8	ROC-AUC chart	Shows separability using one-vs-rest curves.
Dashboard	Figure 9	Dashboard class distribution	Summarizes the end-to-end PCAP demo result.

1 Executive Summary

NetFlow Analyzer is an end-to-end network traffic analysis pipeline that captures PCAP files, reconstructs network flows, extracts explainable Layer 3 and Layer 4 features, trains machine learning models and visualizes predictions in a Streamlit dashboard.

The system addresses a practical limitation of payload-based monitoring: modern traffic is often encrypted, and payload inspection may be expensive, intrusive or unavailable. Instead of relying on payload contents, the pipeline uses behavioral metadata such as flow duration, byte and packet counts, TCP flags, inter-arrival time, estimated RTT, TCP window size and SYN/ACK ratio.

97.96%	96.97%	96.89%	96.76%
Train accuracy	Test accuracy	Weighted F1	CV weighted F1

Main result

The optimized Random Forest reached 0.9697 test accuracy and 0.9689 weighted F1-score. In the final dashboard demo, the system analyzed 4,025 flows and detected 3,999 anomalous flows with 94.16% mean confidence.

2 Problem Definition and Objectives

Network security monitoring must identify malicious or suspicious behavior while handling encrypted traffic, heterogeneous protocols and high traffic volume. Payload-based inspection can provide detailed information, but it is increasingly constrained by encryption and privacy requirements.

This project therefore studies whether protocol-level metadata is sufficient to classify relevant traffic categories in a controlled environment. The goal is not only predictive accuracy, but also technical explainability: each feature should be meaningful from a networking perspective.

Objectives

- Build an isolated and reproducible Docker traffic lab.
- Generate normal HTTP/FTP/DNS traffic and controlled attack traffic.
- Capture PCAP files and reconstruct bidirectional flows.
- Extract L3/L4 features without payload inspection.
- Train K-Means and Random Forest models and evaluate them rigorously.
- Expose the final prediction pipeline through an interactive Streamlit dashboard.

3 System Architecture

The architecture is modular. Traffic generation and capture are separated from feature extraction, model training and dashboard visualization. This separation makes the system easier to test, debug and explain.

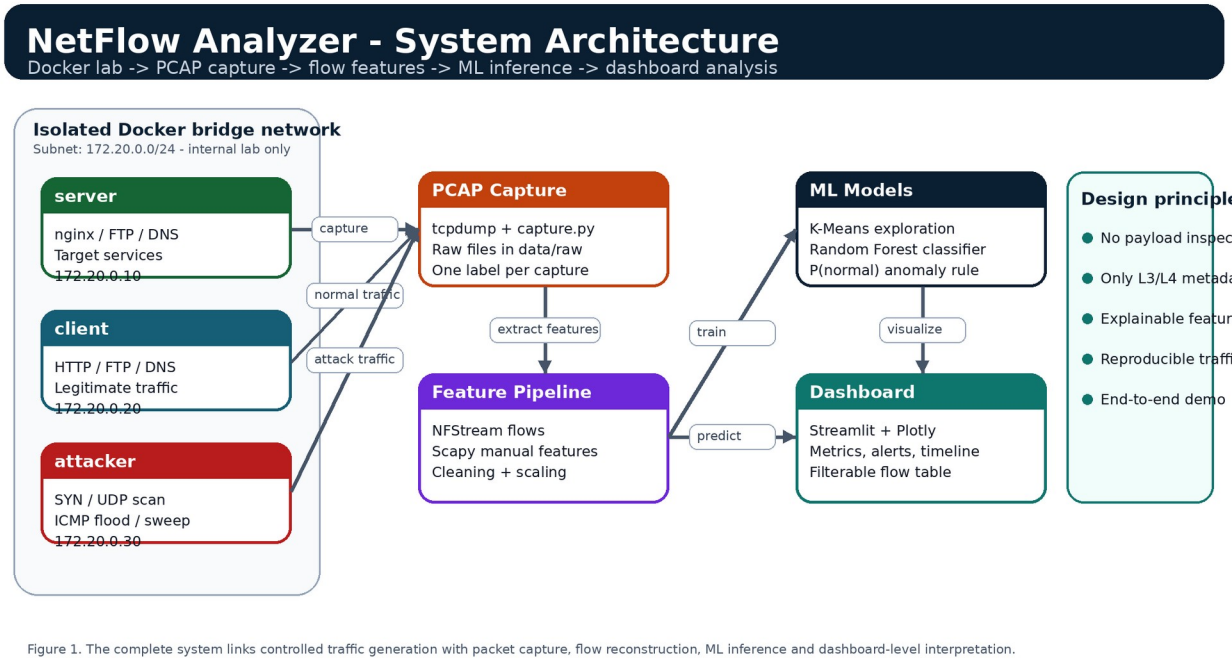

Figure 1. The complete system links controlled traffic generation with packet capture, flow reconstruction, ML inference and dashboard-level interpretation.

Figure 1. System architecture and data flow. The Docker lab generates controlled traffic, PCAP files are processed into L3/L4 features, and predictions are visualized in the dashboard.

Table 3. Docker lab components.

Component	Static IP	Role
server	172.20.0.10	Runs target network services such as HTTP, FTP and DNS.
client	172.20.0.20	Generates legitimate HTTP, FTP and DNS traffic.
attacker	172.20.0.30	Generates ICMP flood, SYN scan, UDP scan and port sweep traffic.

Architecture rationale

Using Docker instead of a real local network makes the experiment reproducible, safer and easier to label. It also avoids collecting private user traffic.

4 Traffic Generation and Capture

Traffic was generated in the isolated Docker bridge network. Legitimate traffic includes heterogeneous application-level behavior, while attack traffic is generated separately to simplify labeling and to validate the detection pipeline under controlled conditions.

Table 4. Generated traffic classes.

Class	Protocol	Tool	Description
normal	HTTP, FTP, DNS	curl, lftp, dig	Mixed browsing-like requests, file transfers and DNS queries.
icmp_flood	ICMP	hping3	High-rate echo traffic used to model flood-like behavior.
syn_scan	TCP	nmap -sS	Half-open TCP scan with incomplete handshakes.
udp_scan	UDP	nmap -sU	UDP port probing with limited response information.
port_sweep	Network discovery	nmap -sn	Host discovery over the Docker lab subnet.

Raw PCAP files are intentionally ignored by Git because they can become large. Their existence, size, packet count and duration are documented in the dataset statistics file.

5 Feature Engineering

Feature Engineering is the core of the project. The system converts low-level packets into interpretable flow-level features that encode network behavior. The most important design choice is that features are extracted without packet payload inspection.

NFStream is used for bidirectional flow reconstruction and base statistics. Scapy is used for manual features that require direct packet-header access, such as inter-arrival time, estimated RTT, TCP window statistics and SYN/ACK ratio.

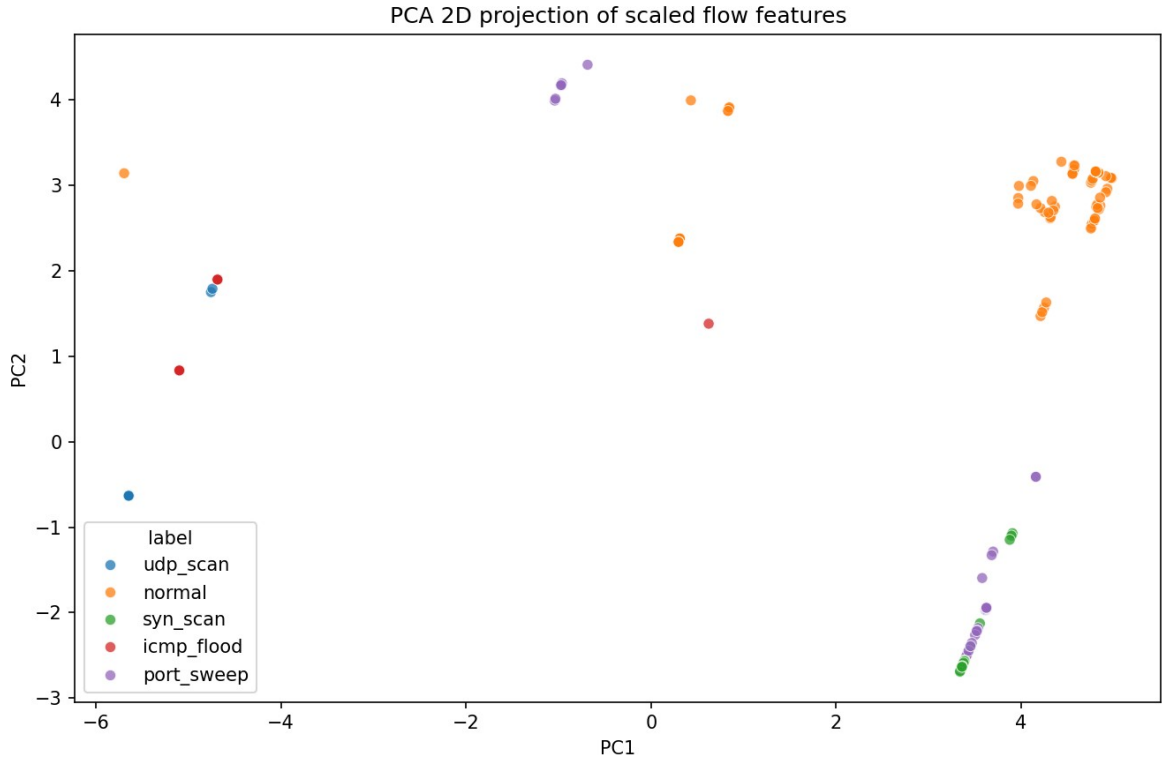

Figure 2. PCA 2D projection of scaled flow features. The class separation indicates that the engineered L3/L4 features encode meaningful network behavior.

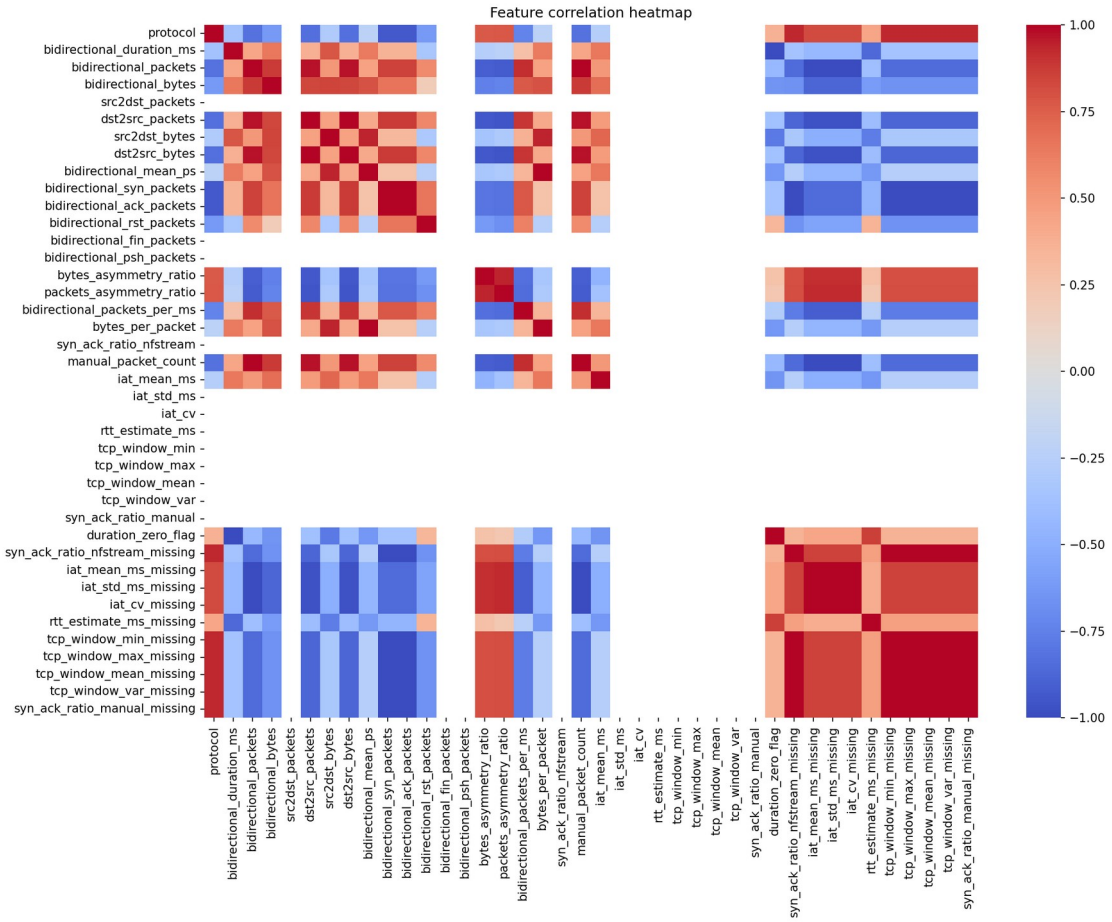

Figure 3. Feature correlation heatmap. Correlation analysis helps identify redundant variables and relationships between protocol-level features.

Table 5. Feature dictionary with protocol-level justification.

Feature group	Calculation	Why it matters
Flow duration	last timestamp - first timestamp	Scans and floods usually produce short flows; FTP/HTTP can last longer.
Packets and bytes	bidirectional packet and byte totals	Measures interaction volume and distinguishes real transfer from control probes.
Directional asymmetry	src2dst vs dst2src bytes/packets	Reconnaissance traffic often sends probes with little response.
Mean packet size	mean packet length within a flow	Control traffic is small; data transfer approaches larger packet sizes.
Packet rate	packets / duration	Flood-like traffic can produce high rate values.
TCP flag counts	SYN, ACK, RST, FIN, PSH counts	Encodes connection state and scan behavior.
SYN/ACK ratio	count(SYN) / count(ACK)	Half-open scans generate SYN-heavy flows without normal completion.
Inter-arrival time	mean and CV of packet gaps	Automated tools often show more regular timing than human/application traffic.
Estimated RTT	SYN-ACK time - SYN time	Defined only when handshake evidence exists; missing RTT is informative.
TCP window stats	min, max, mean, variance	Normal TCP can vary with congestion control; scanners may use fixed values.
Protocol	IP transport protocol	Separates TCP, UDP and ICMP behavior.

Expected behavior by traffic class

Table 6. Expected feature behavior by class.

Traffic class	Expected behavior
Normal HTTP/FTP/DNS	Longer and more variable flows, higher byte volume, valid TCP behavior for HTTP/FTP, DNS over UDP.
ICMP flood	Short control-like flows, no TCP flags, very regular timing and small packet sizes.
SYN scan	Very short TCP flows, SYN/RST behavior, incomplete handshakes and high SYN/ACK ratio.
UDP scan	UDP flows with limited state, low payload and no TCP handshake features.
Port sweep	Short discovery probes; may resemble scan traffic when each flow is analyzed in isolation.

6 Data Preparation and Cleaning

The final dataset was aligned to a fixed feature schema, cleaned and scaled for machine learning. Protocol-specific missing values were handled carefully: for example, RTT is undefined for UDP and ICMP, and a missing RTT in a TCP flow can indicate an incomplete handshake.

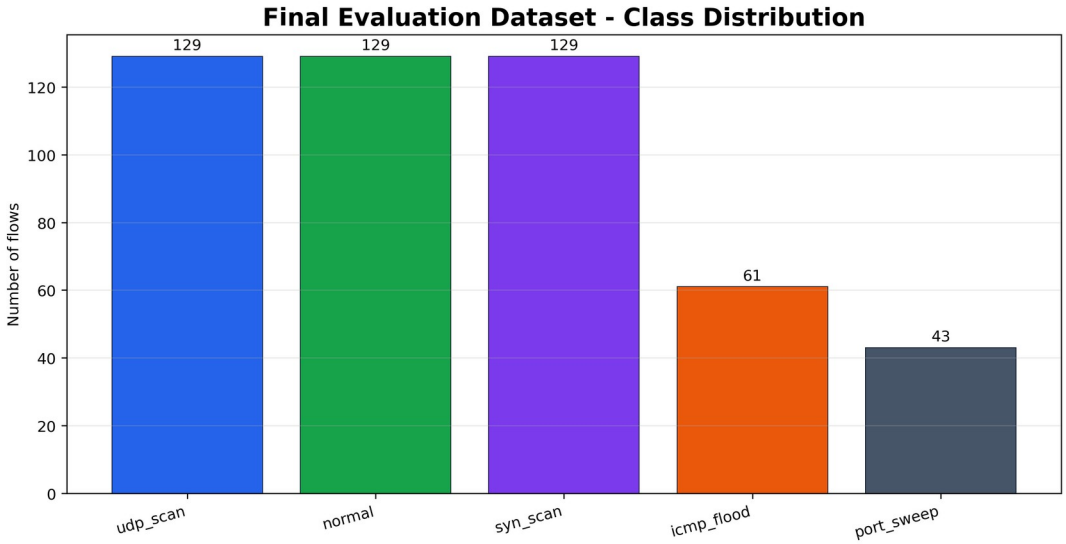

Figure 4. Final evaluation dataset distribution. The dataset is moderately imbalanced, especially for port_sweep and icmp_flood.

Table 7. Final evaluation dataset distribution.

Label	Count	Percentage
udp_scan	129	26.27%
normal	129	26.27%
syn_scan	129	26.27%
icmp_flood	61	12.42%
port_sweep	43	8.76%

Cleaning principle

Missing values were not treated blindly. Some values are missing because the protocol does not support that concept, for example RTT in UDP or ICMP traffic. This makes the cleaning strategy part of the feature-engineering design.

7 Machine Learning Pipeline

The machine learning workflow combines exploratory unsupervised learning with a supervised classifier. K-Means is used to inspect whether flows cluster according to protocol behavior, while Random Forest is used for the final flow classification task.

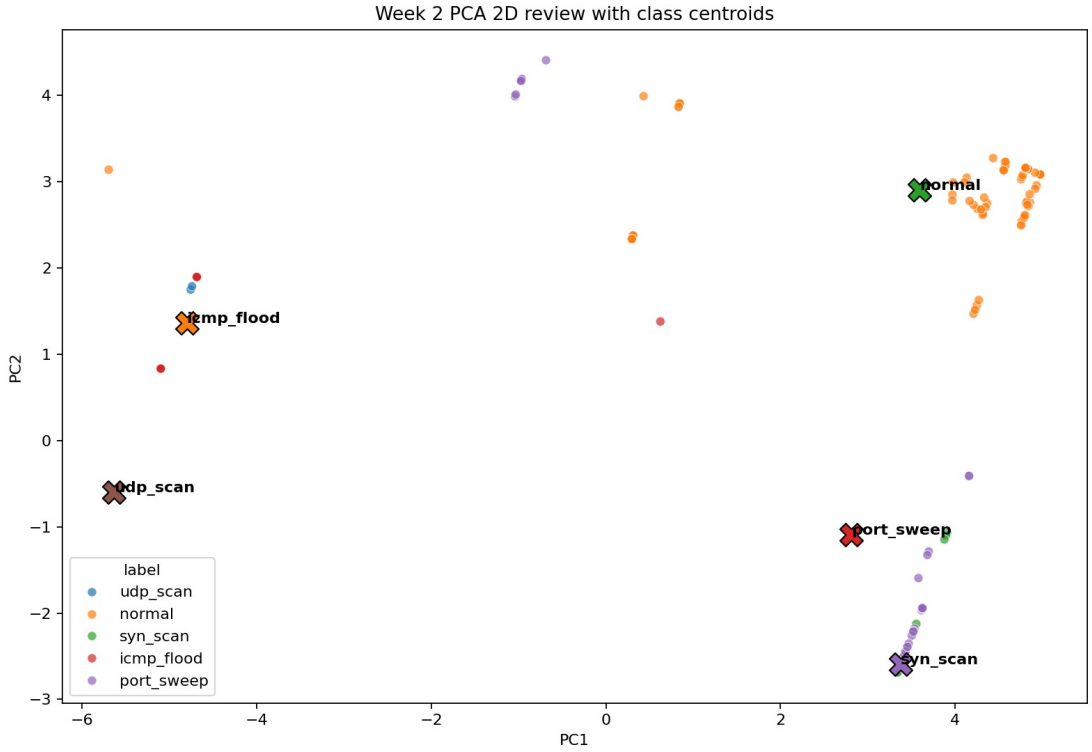

Figure 5. PCA projection with K-Means centroids. The plot provides a compact view of how flows distribute in reduced feature space.

Model artifacts

Table 8. Model artifacts.

Artifact	Purpose
models/kmeans.pkl	Unsupervised clustering model.
models/rf_optimized.pkl	Optimized Random Forest classifier.
models/scaler.pkl	Feature scaler fitted on the training data.
models/label_encoder.pkl	Class label encoder.
models/feature_columns.json	Training feature schema used at prediction time.

Why Random Forest?

Random Forest is appropriate for heterogeneous tabular network features. It is robust to irrelevant variables, reduces overfitting compared with a single Decision Tree and provides feature importance for interpretation.

8 Evaluation Results

The final Random Forest model was evaluated using an 80/20 stratified train/test split and 5-fold stratified cross-validation. The train-test accuracy gap is below 5%, so there is no strong evidence of overfitting.

Table 9. Overall model evaluation metrics.

Metric	Value
Train accuracy	0.9796
Test accuracy	0.9697
Weighted F1-score	0.9689
Mean CV weighted F1	0.9676
CV std weighted F1	0.0198

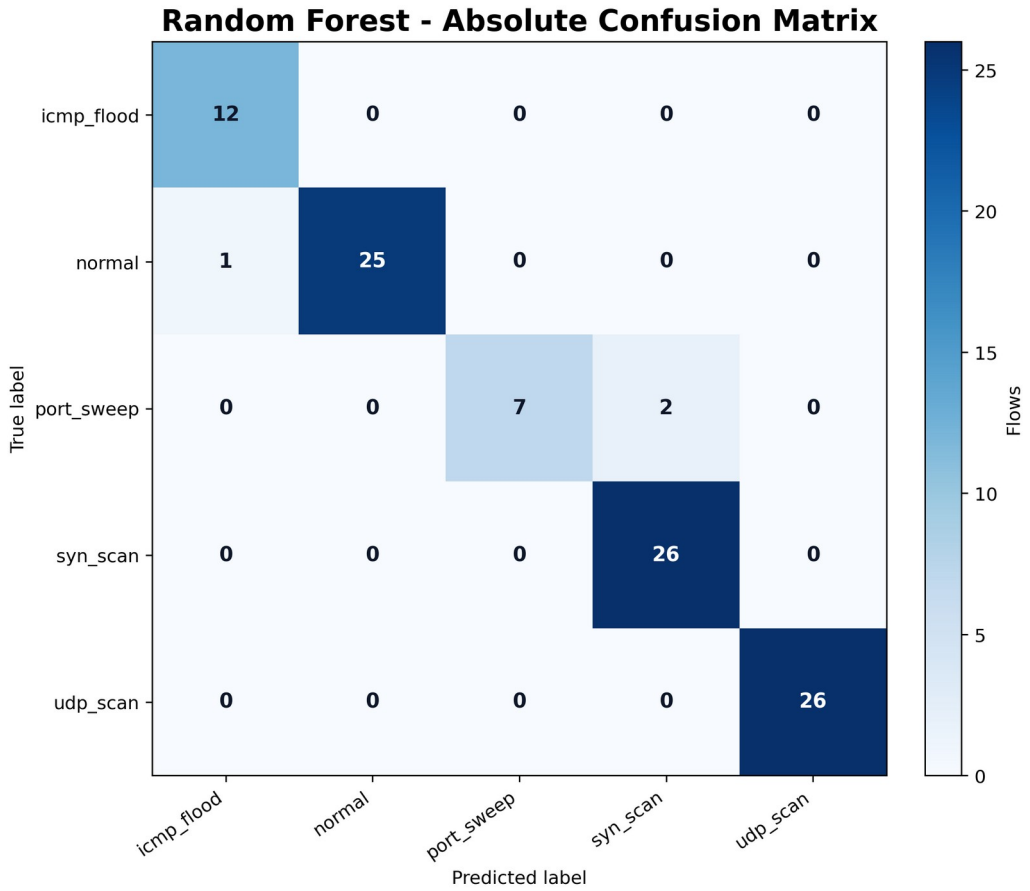

Figure 6. Random Forest confusion matrix. The most relevant errors are two port_sweep flows predicted as syn_scan and one normal flow predicted as icmp_flood.

Table 10. Per-class precision, recall and F1-score.

Class	Precision	Recall	F1-score	Support
icmp_flood	0.9231	1.0000	0.9600	12
normal	1.0000	0.9615	0.9804	26
port_sweep	1.0000	0.7778	0.8750	9
syn_scan	0.9286	1.0000	0.9630	26
udp_scan	1.0000	1.0000	1.0000	26

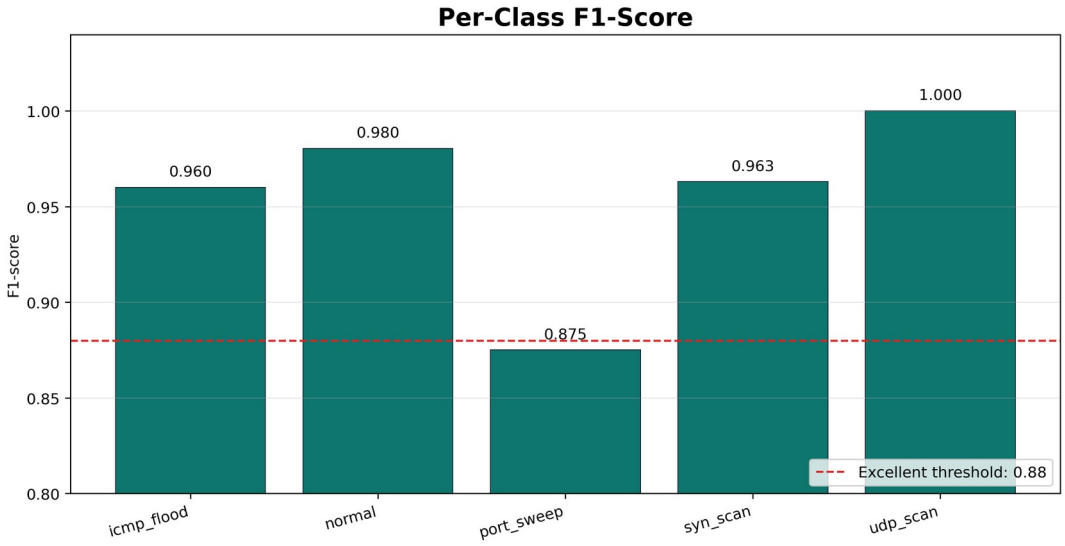

Figure 7. Per-class F1-score. The weakest class is port_sweep, but it remains close to the excellent threshold and all attack flows remain detected as attack-like traffic.

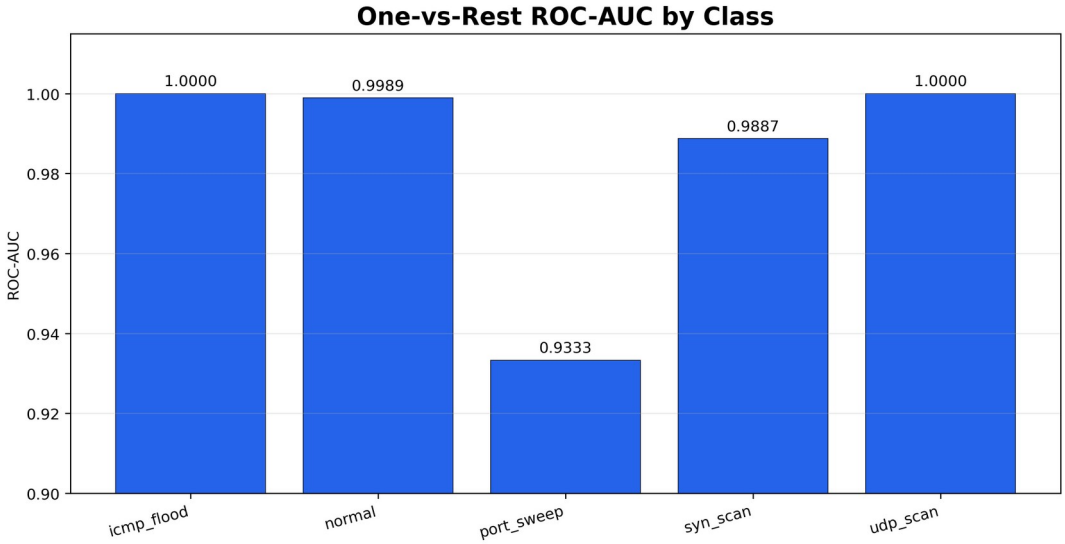

Figure 8. One-vs-rest ROC-AUC scores. UDP scan and ICMP flood are the most separable classes in the current dataset.

Security-oriented interpretation

From a security perspective, false negatives where attack traffic is classified as normal are the most critical errors. In the evaluation split, no attack flow was classified as normal. The two port_sweep errors were predicted as syn_scan, so they remained in an attack-like reconnaissance category.

9 Dashboard and End-to-End Demo

The Streamlit dashboard is the user-facing layer of the project. It lets an analyst upload a PCAP file, run the existing prediction pipeline and inspect classified flows using metrics, charts and a filterable table.

4,025	26	3,999	94.16%
Total flows	Normal flows	Anomalous flows	Mean confidence

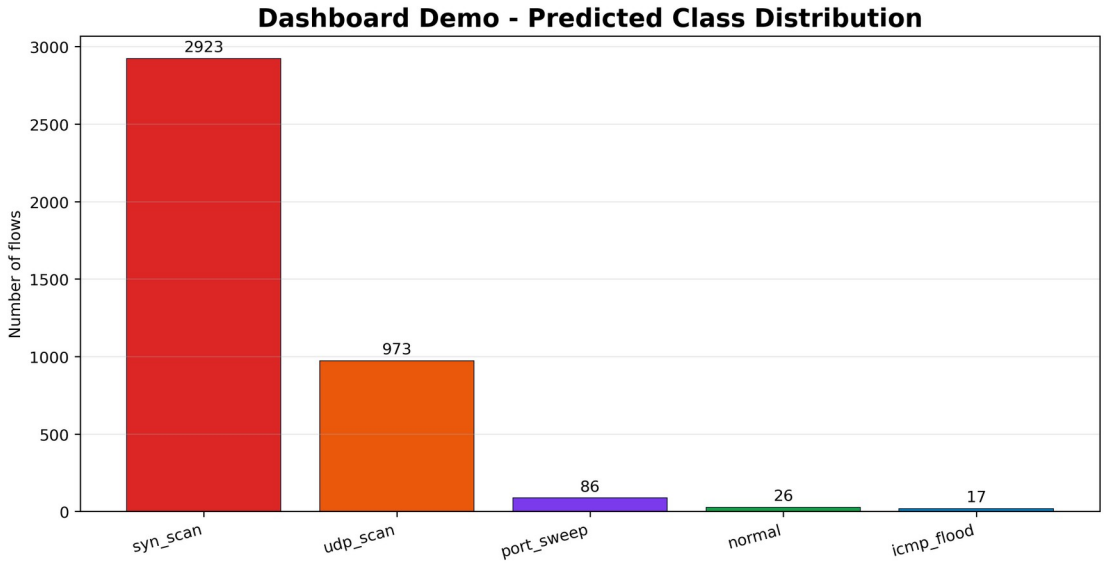

Figure 9. Final dashboard demo class distribution for the mixed PCAP. The most frequent detected attack was syn_scan.

Table 11. Dashboard demo predicted class distribution.

Predicted class	Flows
syn_scan	2923
udp_scan	973
port_sweep	86
normal	26
icmp_flood	17

- The dashboard displays an anomaly alert when malicious or suspicious flows are detected.
- The bytes-versus-duration scatter plot supports visual inspection of traffic behavior.
- The timeline helps identify when attack-like flows start inside the capture.
- The class filter allows the analyst to inspect only flows such as syn_scan or udp_scan.

10 Discussion, Limitations and Future Work

The results show that Layer 3 and Layer 4 metadata can be highly informative for the selected traffic classes. UDP scan and ICMP flood are especially separable because their protocol behavior differs clearly from stateful TCP traffic. SYN scan is also well detected because half-open scanning produces distinctive duration and TCP flag patterns.

The most difficult class is port_sweep, with an F1-score of 0.8750. This is technically coherent: host discovery and scanning probes can resemble other short reconnaissance flows when each flow is analyzed independently.

Limitations

- The dataset is synthetic and generated in a controlled Docker environment.
- The model only covers the traffic classes used during training.
- The system analyzes flows independently and does not yet use sliding time windows.
- The dashboard performs batch PCAP analysis, not real-time streaming.
- IPv6 was not a primary focus in this version.
- Slow low-rate attacks may be harder to detect because they can mimic normal timing and volume.

Future work

- Add temporal aggregation features such as unique destination ports per source and connection attempts per second.
- Evaluate the model on public intrusion datasets such as CICIDS2017 or UNSW-NB15.

- Add near-real-time streaming with Kafka or a rolling PCAP watcher.
- Calibrate Random Forest probabilities to improve confidence interpretation.
- Add automated dashboard tests and report export to PDF or HTML.

11 Conclusions

This project demonstrates that protocol-level metadata can be sufficient to classify and detect several types of controlled network traffic without inspecting packet payloads. The most important lesson is that effective feature engineering connects networking theory with empirical machine learning results.

Features such as flow duration, SYN/ACK ratio, inter-arrival time variability, packet size and TCP flag counts are not arbitrary inputs. They encode real TCP/IP behavior. For example, half-open SYN scans do not behave like complete TCP sessions, and this difference appears directly in duration, flag counts and handshake-related features.

The optimized Random Forest achieved 0.9697 test accuracy and 0.9689 weighted F1-score. The final Streamlit dashboard also proved that the pipeline works end to end, from PCAP upload to anomaly visualization.

Final assessment

NetFlow Analyzer is a successful academic prototype because it combines reproducible traffic generation, explainable feature engineering, strong model evaluation and a functional dashboard demo.

12 Appendix: Visual Assets Included

Table 12. Graphs inserted in the final report.

Figure	Section	Description
Figure 1	Architecture	Used to explain the Docker lab and the full pipeline.
Figure 2	Dataset	Shows class balance and moderate imbalance.
Figure 3	Features	Best visual evidence that engineered features create separable traffic groups.
Figure 4	Features	Shows feature relationships before ML.
Figure 5	ML	Shows cluster structure in reduced feature space.
Figure 6	Evaluation	Shows actual classification errors.
Figure 7	Evaluation	Highlights per-class performance.
Figure 8	Evaluation	Shows separability using one-vs-rest curves.
Figure 9	Dashboard	Summarizes the end-to-end PCAP demo result.

All numerical values in this report are aligned with the repository documentation: model_evaluation.md, dataset_stats.md and week4_day3_end_to_end_demo.md.